Gestion des transactions

Comment garantir la cohérence et la fiabilité tout en assurant des accès concurrents aux données, et plus encore…

Introduction

- La propriété fondamentale d'un SGBD est de garantir l'intégrité des données.
- Mais autoriser les accès concurrents est nécessaire pour un SI.

- 3 objectifs importants liés aux transactions:
  - Garantir un état cohérent
  - Garantir la pérennité des données
  - Garantir la performance et donc la concurrence d'accès

Un exemple très simplifié

- On considère un système de gestion de stades basés sur trois tables :
  Stade(nomStade, NbPlaces),
  Match(Affiche, NomStade, Prix, PlacesPrises)
  Client(NoClient, Solde).

- et disposant d'une fonctionnalité de réservation de places pour un client.

Exemple: Réservation de places

```
Places (Client C, Match M, NbPlaces N)
begin
        Lire le match M (tuple de la table match)
        Lire le stade S (tuple de la table stade)

        if ( (S.nbplaces – M.PlacesPrises) >= N )
        begin
                Lire le client C (tuple de la table client)
                M.PlacesPrises += N
                C.solde -= N * M.Prix
                Ecrire le match M (tuple modifié de la table match)
                Ecrire le client C (tuple modifié de la table client)
        end
end
```

Exemple: scénario concurrent

- Soit l'histoire suivante représentant deux exécutions concurrentes de la fonction de réservation pour le même match et le même client (*scalper ?*):
  - T1=Places(C, M, 100)
  - T2=Places(C, M, 200)

T1	L(M)
T1	L(S)
T1	L(C)
T1	E(M)
T2	L(M)
T2	L(S)
T2	L(C)
T2	E(M)
T1	E(C)
T2	E(C)

- Si on suppose que le stade dispose de 1000 places, sans résa pour le match au début de l'exécution: Quel est l'état de la base à la fin ?

Trace de l'histoire

T1	L(M)	=> 0 places réservées
T1	L(S)	=> 1000 places dans le stade
T1	L(C)	=> soit 5000 le solde du client
T1	E(M)	=> 100 places réservées par le client
T2	L(M)	=> 100 places réservées
T2	L(S)	=> 1000 places dans le stade
T2	L(C)	=> soit 5000 le solde du client
T2	E(M)	=> 300 places réservées par le client
T1	E(C)	=> 5000 – 100*prix
T2	E(C)	=> 5000 – 200*prix

- 300 places ont été réservées, mais 200 ont été comptabilisées au client !
- Il faut mettre en place des contraintes sur ces opérations concurrentes.

Notion de transaction

- **Une transaction est une opération ou une suite d'opérations qui accèdent et/ou mettent à jour à jour des données de la base.**

  C'est une unité logique de travail.

Ex: une suite de commande SQL peut former une transaction composée de plusieurs opérations.

2 requêtes / 2 opérations	1 requête / 2 opérations combinées
Select count(*) from T1	
	Update T1
Select count(*) from T1	Set attr=select count(*) from T2

Fonctionnement d'une transaction

- Une transaction est constituée de 2 types d'opérations:
  - Lecture, notée lire(x) ou L(x) ou R(x)
  - Ecriture, notée ecrire(x) ou E(x) ou W(x)
- Une transaction possède 2 fins possibles
  - Achèvement avec succès, elle est validée ou confirmée (committed)
  - En échec, elle est annulée (aborted). L'annulation consiste à replacer la base dans le dernier état connu avant l'exécution de la transaction.

Notion de concurrence

- La concurrence d'accès dans un environnement multi-utilisateurs permet une meilleure utilisation des ressources.
  - Simultanéité des accès
  - Mais compétition sur les données
- Les opérations d'E/S sont interfoliées pour assurer un accès concurrent aux données.

Propriétés ACID assurées par un système de gestion des transactions

- **Atomicité**: "tout ou rien". Toutes les opérations d'une transaction doivent apparaître dans la base, ou aucune. L'annulation d'une transaction fait disparaitre toutes actions modifiantes.

Ex: La méthode Places forme une transaction. Un dysfonctionnement lors de l'exécution de la méthode => incohérence de la base sans l'atomicité.

…

Lire le client C (tuple de la table client)

M.PlacesPrises += N

C.solde -= N * M.Prix

Ecrire le match M (tuple modifié de la table match)

Ecrire le client C (tuple modifié de la table client)

Propriétés ACID assurées par un système de gestion des transactions

- **Cohérence**: une transaction fait passer la BD d'un état cohérent à un autre état cohérent. Une transaction préserve la cohérence de la base. Prise en compte des contraintes de schéma et de métier.

Ex: les opérations de la transaction ne sont possible qu'à la condition métier de « places suffisantes ».

```
if ( (S.nbplaces – M.PlacesPrises) >= N )
…
```

Propriétés ACID assurées par un système de gestion des transactions

- **Isolation**: les transactions en concurrence s'exécutent de manière indépendante les unes des autres. Pour chaque paire de transactions Ti et Tj, si Ti isolée alors Ti
  - Voit Tj comme terminée avant qu'elle commence
  - Considère que Tj commence une fois qu'elle termine.
  - Sinon les problèmes commencent si elles travaillent sur les mêmes données…

Ex: voir exemple d'intro.

T1 pas terminée quand T2 commence.
T1 et T2 écrivent en concurrence la même donnée sans aucune contrainte d'écriture / lecture => problème !!!

Propriétés ACID assurées par un système de gestion des transactions

- **Durabilité**: les effets d'une transaction complètement achevée et validée sont inscrits de manière durable dans la BD. Ils ne peuvent être altérés par une défaillance.

Ecrire le match M (tuple modifié de la table match)

Ecrire le client C (tuple modifié de la table client)

Ces deux écritures doivent persister si une transaction est validée, même si une panne intervient.

Un SGBD possède donc un système de contrôle de concurrence et de reprise après panne capable de respecter ces propriétés.

Problèmes liés à la concurrence

- L'entrelacement (interfoliage) des opérations de plusieurs transactions est un **ordonnancement** lorsque l'ordre des opérations de chaque transaction en concurrence est préservée.
- Des transactions a priori correctes peuvent produire des résultats incohérents en fonction de l'ordonnancement choisi.
- Problèmes pouvant apparaître:
  - Lecture sale
  - Perte de mise à jour
  - Lecture non reproductible
  - Lecture fantôme

Perte de mise à jour (O1)

T1	T2
	Début transaction
Début transaction	
lire(solde client 1)	lire(solde client 1)
solde=solde-10	solde=solde+100
	ecrire(solde client 1)
ecrire(solde client 1)	validation
Validation	

- Une opération se trouve écrasée par une autre.
  - T1 et T2 démarrent presque au même temps
  - T1 et T2 lisent la même valeur de solde
  - La dernière écriture de T1 écrase celle de T2

<u>solde=solde+100 est perdue</u>

Lecture sale (O2)

T1	T2
	Début transaction
	lire(solde client 1)
	solde=solde+100
	ecrire(solde client 1)
Début transaction	
lire(solde client 1)	
solde=solde-10	
ecrire(solde client 1)	
Validation	
	…
	Annulation

- Une transaction lit les données écrites par une autre transaction non encore validée (O1 => O2)
  - T2 modifie la valeur de solde
  - T1 lit la valeur de solde modifiée et réalise sa tâche en se basant sur cette valeur
  - T2 annule entre-temps ces propres opérations.

<u>solde=solde+100-10</u> (**incohérent**)

Lecture non reproductible (O3)

T1	T2
Début transaction	Début transaction
lire(solde client 1)	lire(solde client 1)
solde=solde-10	solde=solde+100
…	ecrire(solde client 1)
lire(solde client 1)	validation
…	
Validation	

- Une transaction voit les modifications des tuples validées.
- Une transaction relit un enregistrement lu précédemment mais modifié, entre-temps, par une autre transaction.
  - T1 lit la valeur de solde
  - T2 modifie la valeur de solde et valide
  - T1 relit la valeur de solde qui est différente de sa première lecture.

<u>solde=100</u>

Lecture non reproductible (O3)

T1	T2
	Début transaction
Début transaction	lire(solde client 1)
select avg(solde) from client	solde=solde+100
solde=solde-10	ecrire(solde client 1)
…	validation
select sum(solde) from client	
…	
Validation	

Problème plus évident avec des agrégations !

- Moyenne et Somme donne des résultats incohérents dans une synthèse de résultats:
  - La valeur de solde du client 1 évolue entre les deux opérations d'aggrégation de T1.
  - Moyenne et Somme sont calculés sur des ensembles de valeur différents durant la vie de T1
- Le résultat de T1 aurait-il été cohérent avec deux compteurs ?
  - Select count(*) from client

Lecture fantôme (O4)

T1	T2
Début transaction	Début transaction
…	select * from client
select count(*) from client	insert into client (101, 10000)
…	…
	insert into client (102, 5000)
select count(*) from client	Validation
…	
Validation	

- Une requête exécutée plusieurs fois peut contenir de nouveaux enregistrements (et modifie donc le résultat):
  - T1 compte le nombre de client
  - T2 ajoute deux clients
  - T1 recompte le nombre de clients et trouve un client de plus que le résultat précédent (le client 102)

Lecture fantôme (O4)

T1	T2
Début transaction	Début transaction
…	select * from client
select sum(solde) from client	insert into client (101, 10000)
…	…
select count(*) from client	insert into client (102, 5000)
…	
Validation	Validation

- Le problème devient évident avec deux opérations d'agrégation différentes:
  - SUM se base sur un ensemble de tuples différents de COUNT
  - Si le résultat des deux requêtes est utilisée dans un calcul à l'intérieur d'un contrôleur: incohérence !

Gestion de transactions en pratique

- Elle repose sur 2 commandes
  - Validation = commande Commit
  - Annulation = commande Rollback
- Valider une transaction: la commande **Commit** indique la fin d'une transaction et le début d'une autre. Une transaction est l'ensemble des opérations entre 2 commit. => **Durabilité**
- Annuler une transaction: la commande **RollBack** permet d'annuler toutes les opérations d'une transaction. => **Atomicité**
  La notion de "savepoint" permet d'annuler une partie d'une transaction, mais n'annuler pas la transaction.
  - savepoint Nom_du_point_de_sauvegarde
- Mode AutoCommit: Un SGBD est parfois en mode **autocommit**, toutes les opérations sont systématiquement validées. Aucune transaction n'est possible dans ce mode (1 transaction = 1 opération).
  - Set autocommit on/off pour basculer d'un mode à l'autre

Protocole de verrouillage

- Lorsqu'une transaction accède à une donnée, un verrou est susceptible de bloquer l'accès aux autres transactions.
- On distingue 2 types de verrous:
  - Partagé: si une transaction dispose d'un verrou partagé sur une donnée, elle peut la lire mais pas la modifier = verrou en lecture (VL).

    Plusieurs transactions peuvent se partager un même verrou en lecture.
  - Exclusif: si une transaction dispose d'un verrou exclusif sur une donnée, elle peut la lire et la modifier = verrou en écriture (VE).

    Une seule transaction peut obtenir un verrou Exclusif, les autres transactions doivent attendre la libération du verrou pour accéder à la donnée (en posant un nouveau verrou).

Protocole de verrouillage

- Toute transaction devant accéder à une donnée doit obtenir un verrou sur la donnée: soit VE(x) , soit VL(x)
- Si la donnée est déjà verrouillée par une autre transaction avec un verrou non compatible (seul VL est compatible avec VL) la transaction doit attendre la libération du verrou.
- Une transaction conserve un verrou tant qu'elle ne le libère pas:
  - Explicitement avec une commande de libération notée L(x).
  - Implicitement avec une terminaison de transaction: validation (commit) ou annulation (abort/rollback).

- **Insuffisant pour garantir la cohérence des données…**

Transaction et isolation

Quatre niveaux d'isolation sont définis pour relâcher ou consolider les Pbs de lecture sale, perte de mise à jour, lecture non reproductible et lecture fantôme

- Chaque transaction peut choisir son niveau d'isolation
- Le niveau d'isolation modifie la visibilité des écritures des autres transactions: isolation +/- relâchée
- En pratique chaque niveau pilote une pose de verrous spécifiques (type de verrou, granularité, versioning)

Niveaux d'isolation 1/2

- Serializable
  - La transaction suit le protocole 2PL strict
  - La transaction ne voit que les modifications validées au début de son exécution et ses propres modifications => synchronisation nécessaire à sa validation.
  - Une même requête (ré-exécutée) donnera le même résultat car les modifications sont bloquées pour les autres transactions: Lecture fantôme OK.
- Repeatable read
  - Un même enregistrement peut être lu plusieurs fois sans modification possible: pas de modifications possibles entre deux « read » sur les données lues
  - Verrous partagés maintenus
  - Pas de 2PL: Fantômes possibles !

Niveaux d'isolation 2/2

- Read commited
  - Chaque requête de la transaction ne voit que les modifications validées avant son exécution.
  - L'état de la base peut changer entre deux requêtes.
  - fantômes et lecture non reproductibles possibles
  - Perte de mise à jour possible
  - Verrous partagé relâchés après lecture
- Read uncommitted
  - La transaction peut lire des objets écrits par une autre transaction (lecture non bloquante)
  - lectures sales possibles

Niveaux d'isolation: résumé

	Lecture sale	Lecture non reproductible	fantôme
Read Uncommited	Oui	Oui	Oui
Read commited	Non (mais perte de MAJ)	Oui	Oui
Repeatable Read	Non	Non	Oui
Serializable	Non	Non	Non